Imam Abdulrahman Bin Faisal University
College of Computer Science and Information Technology

ARTI 404 — Natural Language Processing

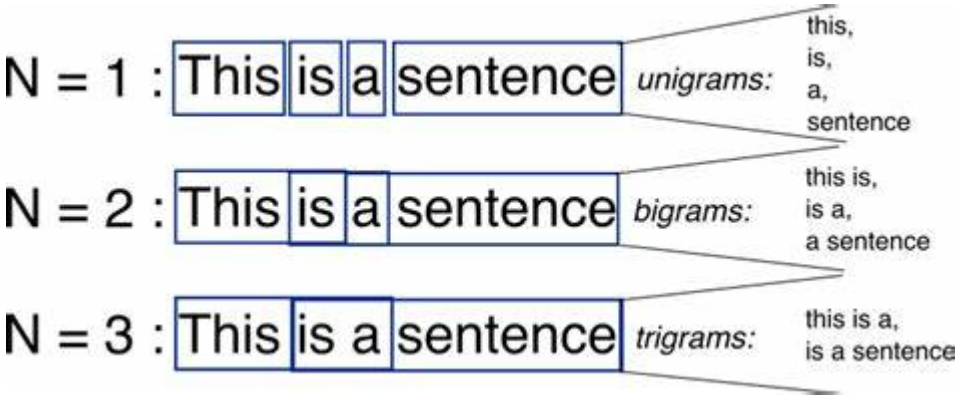
Lab 3 — N-Grams

Student Name	Ahmed Albuainain
Student ID	2240006128
Section	7MA1

N-Grams

It's a probabilistic model that's trained on a corpus of text. Such a model is useful in many NLP applications including speech recognition, machine translation and predictive text input. An N-gram model is built by counting how often word sequences occur in corpus text and then estimating the probabilities.

Types of N-Grams



Use-cases of N-Grams

- Auto completion of sentences
- Auto spell check
- Voice-based personal assistant bots

Unigram

Unigram is an n-gram consisting of a single item of a sequence.

- It is commonly used to calculate the probability of finding a word in an article.

$$
P(A) = \frac{\text{count of } A}{\text{count of all words}}
$$

```
import nltk
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

unigrams = ngrams(word_tokenize(string), 1)

for item in unigrams:
    print(item)
```

```
('moses',)
('supposes',)
('his',)
('toeses',)
('are',)
('roses',)
('but',)
('moses',)
('supposes',)
('erroneously',)
```

Calculating probabilities in unigram models

```
### Calculate counts of each word

string = "moses supposes his toeses are roses but moses supposes erroneously"

unique_words = set(word_tokenize(string))

words_frq = []
for uniques in unique_words:
    frq = string.count(uniques)
    words_frq.append((uniques, frq))

words_frq
```

```
[('his', 1),
 ('are', 1),
 ('supposes', 2),
 ('roses', 1),
 ('but', 1),
 ('erroneously', 1),
 ('moses', 2),
 ('toeses', 1)]
```

```
### Calculate probabilities:

probab = []

for count in range(len(words_frq)):
    pro = words_frq[count][1]/len(word_tokenize(string))
    probab.append((words_frq[count][0],pro))

print('The probabilities are: ', probab)
```

```
The probabilities are:  [('his', 0.1), ('are', 0.1), ('supposes', 0.2), ('roses', 0.1),
('but', 0.1), ('erroneously', 0.1), ('moses', 0.2), ('toeses', 0.1)]
```

Bigram

Bigram an n-gram consisting of two items of a sequence
- Two words that are used together to mean something specific

$$
P(B \mid A) = \frac{\text{Count of }(AB)}{\text{Count of }(A)}
$$

Method 1

```
import nltk
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

bigrams = nltk.bigrams(word_tokenize(string))
for i in bigrams:
    print(i)
```

```
('moses', 'supposes')
('supposes', 'his')
('his', 'toeses')
('toeses', 'are')
('are', 'roses')
('roses', 'but')
('but', 'moses')
('moses', 'supposes')
('supposes', 'erroneously')
```

Method 2

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

bigrams = ngrams(word_tokenize(string),2)

for i in bigrams:
    print(i)
```

```
('moses', 'supposes')
('supposes', 'his')
('his', 'toeses')
('toeses', 'are')
('are', 'roses')
('roses', 'but')
('but', 'moses')
('moses', 'supposes')
('supposes', 'erroneously')
```

Trigram

Trigram an n-gram consisting of three items of a sequence
- Three words that are used together to mean something specific

Method 1

```
import nltk
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

trigrams = nltk.trigrams(word_tokenize(string))

for i in trigrams:
    print(i)
```

```
('moses', 'supposes', 'his')
('supposes', 'his', 'toeses')
('his', 'toeses', 'are')
('toeses', 'are', 'roses')
('are', 'roses', 'but')
('roses', 'but', 'moses')
('but', 'moses', 'supposes')
('moses', 'supposes', 'erroneously')
```

Method 2

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

trigrams = ngrams(word_tokenize(string),3)

for i in trigrams:
    print(i)
```

```
('moses', 'supposes', 'his')
('supposes', 'his', 'toeses')
('his', 'toeses', 'are')
('toeses', 'are', 'roses')
('are', 'roses', 'but')
('roses', 'but', 'moses')
('but', 'moses', 'supposes')
('moses', 'supposes', 'erroneously')
```

What is padding in NLP?

Padding is a technique used to ensure that all input sequences have the same length. This is necessary because neural networks require inputs that have the same shape and size. However, when we pre-process and use texts as inputs for our model, not all sentences have the same length. In other words, some sentences are longer or shorter than others. We need to have inputs with the same size, and this is where padding comes in.

Padding is a special form of masking where the masked steps are at the start or the end of a sequence. Padding comes from the need to encode sequence data into contiguous batches: in order to make all sequences in a batch fit a given standard length.
The padding is added before splitting the sentence.

n-1 padding is added.

```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize


string = "moses supposes his toeses are roses but moses supposes erroneously"

print('------------------- Padding in Bigrams ------------------------------')
list(ngrams(word_tokenize(string), pad_left=True, left_pad_symbol="<s>",
                                pad_right=True, right_pad_symbol="</s>", n=2))
```

```
------------------- Padding in Bigrams ------------------------------
```

```
[('<s>', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'his'),
 ('his', 'toeses'),
 ('toeses', 'are'),
 ('are', 'roses'),
 ('roses', 'but'),
 ('but', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'erroneously'),
 ('erroneously', '</s>')]
```

```
print('------------------- Padding in Trigrams ------------------------------')
list(ngrams(word_tokenize(string), pad_left=True, left_pad_symbol="<s>",
                                pad_right=True, right_pad_symbol="</s>", n=3))
```

```
------------------- Padding in Trigrams ------------------------------
```

```
[('<s>', '<s>', 'moses'),
 ('<s>', 'moses', 'supposes'),
 ('moses', 'supposes', 'his'),
 ('supposes', 'his', 'toeses'),
 ('his', 'toeses', 'are'),
 ('toeses', 'are', 'roses'),
 ('are', 'roses', 'but'),
 ('roses', 'but', 'moses'),
 ('but', 'moses', 'supposes'),
 ('moses', 'supposes', 'erroneously'),
 ('supposes', 'erroneously', '</s>'),
 ('erroneously', '</s>', '</s>')]
```

Alternatively, We can get rid of the parameters by using

> pad_both_ends method in NLTK

Note the n argument, that tells the function we need padding for bigrams.

```
from nltk.lm.preprocessing import pad_both_ends
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

list(ngrams(pad_both_ends(word_tokenize(string), n=2), n=2))
```

```
[('<s>', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'his'),
 ('his', 'toeses'),
 ('toeses', 'are'),
 ('are', 'roses'),
 ('roses', 'but'),
 ('but', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'erroneously'),
 ('erroneously', '</s>')]
```

Training the Maximum Likelihood Model

Maximum likelihood estimation estimates the model parameters such that the probability is maximized.

- Get the requisite n_grams frequency counts from a corpus.
- Normalize them to a 0-1 range.
- Build the vocabulary (of unique words): to create this vocabulary we need to pad our sentences (just like for counting ngrams) and then combine the sentences into one flat stream of words.

To do the above steps, we need to create an Ngram and add the padding then flatten the list. All these steps are requested for pre-processing to train the MLE model. However, we can use padded_everygram_pipeline() in NLTK to do all the steps in one call.

The padded_everygram_pipeline() will return:
1- **ngrams_gen:** Each sentence becomes a list of n-grams with padding.
2- **vocab_gen:** A flat stream of all tokens, with start and end symbols included

References:

https://www.nltk.org/api/nltk.lm.html

https://www.kaggle.com/code/alvations/n-gram-language-model-with-nltk

```
from nltk.lm.preprocessing import padded_everygram_pipeline
from nltk.util import ngrams
from nltk.tokenize import word_tokenize
from nltk.lm import smoothing

string = "moses supposes his toeses are roses but moses supposes erroneously"

#Create Bigram Language Model
train, vocab = padded_everygram_pipeline(2, [word_tokenize(string)])
```

```
from nltk.lm import MLE
lm = MLE(2)
lm.fit(train, vocab) #Train the Maximum Likelihood Model
```

Calculating Probabilities

Here's how you get the score for a word given some preceding context. For example we want to know what is the chance that "b" is preceded by "a".

```
print(lm.score('supposes', ['moses'])) #Calculate Bigram Probabilities for
P('moses'|'supposes')
print(lm.score('moses', ['supposes'])) #Calculate Bigram Probabilities for
P('supposes'|'moses') - Returns 0 because unseen
```

```
1.0
0.0
```

Evaluating n_grams models using preplexity

Preplexity is a measurement of how well a probability model (like a language model) predicts a sample.

It's the probability of the test set normalized by the number of words.

A `lower` perplexity means:

- The model is less "perplexed" by the data.
- The model assigns higher probabilities to the actual words in the test set.
- Therefore, better performance.

A `higher perplexity` = more uncertainty = worse model.

```
test = [('moses', 'supposes'), ('toeses','are')]
print(lm.perplexity(test)) #Perplexity for bigram model
```

```
1.0
```

Task: Generating tweets using n_grams

In this Task, we will build a MLE model with n-gram to generate tweets.

The agenda is:
1. load the data:
https://www.kaggle.com/datasets/adizafar/large-random-tweets-from-pakistan

2. pre-processing the data:
   * Remove hashtags
   * Remove RT
   * Remove any website
   * Remove any mentions
   * Remove emojis
3. build MLE model with n-gram (Bi-gram)
4. evaluate the model
5. calculate the probability of the bigram: (pakistan is)
6. calculate the preplexity of the word: (pakistan)

before we start we need to install library to handle the emojis in tweets

> pip install emoji

```
!pip install emoji
```

```
Requirement already satisfied: emoji in c:\users\ahmad\miniconda3\lib\site-packages (2.16.0)
```

```python
import kagglehub, os, glob, re
import pandas as pd

# 1. Load the data
tweets_path = kagglehub.dataset_download("adizafar/large-random-tweets-from-pakistan")
print("Path:", tweets_path)
print("Files:", os.listdir(tweets_path))

csv_file = glob.glob(os.path.join(tweets_path, "*.csv"))[0]
data = pd.read_csv(csv_file, encoding="utf-8", encoding_errors="ignore")

print("\nShape:", data.shape)
print("Columns:", list(data.columns))
data.head()
```

```
Path: C:\Users\ahmad\.cache\kagglehub\datasets\adizafar\large-random-tweets-from-
pakistan\versions\2
Files: ['Random Tweets from Pakistan- Cleaned- Anonymous.csv']

Shape: (202202, 7)
Columns: ['Unnamed: 0', 'created_at_tweet', 'full_text', 'retweet_count', 'favorite_count',
'reply_count', 'location']
```

```
C:\Users\ahmad\AppData\Local\Temp\ipykernel_23800\2628300762.py:10: DtypeWarning: Columns (5)
have mixed types. Specify dtype option on import or set low_memory=False.
  data = pd.read_csv(csv_file, encoding="utf-8", encoding_errors="ignore")
```

	Unnamed: 0	created_at_tweet	full_text	retweet_count	favorite_count	reply_count	location
0	0	Wed Jul 03 07:20:15 +0000 2019	تیرا لیڈر میرا لیڈر نواز شریف ❤️نواز شریف کیا...	226	1097	127	Punjab, Pakistan
1	1	Fri Jul 02 02:50:53 +0000 2021	Happy birthday to my brother n boss , May you ...	54	273	9	Punjab, Pakistan
2	2	Fri Jul 02 12:12:53 +0000 2021	❤️❤️	50	131	13	Punjab, Pakistan
3	3	Tue Oct 15 22:29:16 +0000 2019	`suspicious °jikook au jimin'in yaşadığı kasab...	522	2134	86	İstanbul, Türkiye
4	4	Wed May 13 04:34:31 +0000 2020	Speaking of @SpiderMan... 😂 https://t.co/uGJ...	74	535	11	Follow the ATP Tour ➡

```
import emoji

# The column that holds the tweet text
text_col = 'full_text'
print("Text column:", text_col)

def clean_tweet(text):
    text = str(text)
    text = re.sub(r"\bRT\b", " ", text)            # Remove RT
    text = re.sub(r"http\S+|www\.\S+", " ", text)  # Remove any website
    text = re.sub(r"@\w+", " ", text)              # Remove any mentions
    text = re.sub(r"#\w+", " ", text)              # Remove hashtags
    text = emoji.replace_emoji(text, replace="")   # Remove emojis
    text = re.sub(r"\s+", " ", text).strip().lower()
    return text

data['clean'] = data[text_col].apply(clean_tweet)
data = data[data['clean'].str.split().str.len() >= 2].reset_index(drop=True)

print("Tweets after cleaning:", len(data))
print("\nBefore:", data[text_col][0])
print("After :", data['clean'][0])
data[[text_col, 'clean']].head()
```

```
Text column: full_text
Tweets after cleaning: 171454

Before: ؟؟؟ ہے بھی پکا آپکا کیا ❤️شریف نواز شریف نواز لیڈر میرا لیڈر تیرا @MaryamNSharif
@Tanverhussan @SenPervaiz https://t.co/mDyOGs3sXv
After : ؟؟؟ ہے بھی پکا آپکا کیا شریف نواز شریف نواز لیڈر میرا لیڈر تیرا
```

	full_text	clean
0	❤️شریف نواز شریف نواز لیڈر میرا لیڈر تیرا کیا...	شریف نواز شریف نواز لیڈر میرا لیڈر تیرا کیا آپ...
1	Happy birthday to my brother n boss , May you ...	happy birthday to my brother n boss , may you ...
2	`suspicious °jikook au jimin'in yaşadığı kasab...	`suspicious °jikook au jimin'in yaşadığı kasab...
3	Speaking of @SpiderMan... 😂 https://t.co/uGJ...	speaking of ...
4	Alexa, skip to 2021	alexa, skip to 2021

```python
from nltk.tokenize import word_tokenize
from nltk.lm.preprocessing import padded_everygram_pipeline
from nltk.lm import MLE

# Tokenize every cleaned tweet
tokenized = [word_tokenize(t) for t in data['clean']]
print("Number of tokenized tweets:", len(tokenized))
print("Example:", tokenized[0])

# Build the bigram MLE model
train, vocab = padded_everygram_pipeline(2, tokenized)

lm = MLE(2)
lm.fit(train, vocab)

print("Vocabulary size:", len(lm.vocab))
```

```
Number of tokenized tweets: 171454
Example: ['','آپکا' ,'کیا' ,'شریف' ,'نواز' ,'شریف' ,'نواز' ,'لیڈر' ,'میرا' ,'لیڈر' ,'تیرا'
؟؟؟ھے' ,'بھی'']
Vocabulary size: 160561
```

```python
from nltk.util import bigrams
from nltk.lm.preprocessing import pad_both_ends

# 4. Evaluate the model using perplexity on tweets from the corpus
for sent in tokenized[:3]:
    test_bigrams = list(bigrams(pad_both_ends(sent, n=2)))
    print(f"Perplexity = {lm.perplexity(test_bigrams):.4f}   <-  {' '.join(sent[:10])}
...")

# An unseen bigram gets zero probability in MLE (no smoothing) -> infinite perplexity
print("\nUnseen bigram perplexity:", lm.perplexity([('qwerty', 'zxcvb')]))
```

```
Perplexity = 114.8610   <-  تیرا لیڈر میرا لیڈر نواز شریف نواز شریف کیا آپکا ...
Perplexity = 90.7513   <-  happy birthday to my brother n boss , may you ...
Perplexity = 6.5762   <-  ` suspicious °jikook au jimin'in yaşadığı kasabada aniden başlayan
cinayetler ...

Unseen bigram perplexity: inf
```

```python
# 5. Probability of the bigram ("pakistan is")
print("P(is | pakistan) =", lm.score('is', ['pakistan']))
print("Count('pakistan') =", lm.counts['pakistan'])
print("Count('pakistan is') =", lm.counts[['pakistan']]['is'])

# 6. Perplexity of the word ("pakistan")
print("\nPerplexity of 'pakistan' =", lm.perplexity([('pakistan',)]))
```

```
P(is | pakistan) = 0.05018682687376365
Count('pakistan') = 13649
Count('pakistan is') = 685

Perplexity of 'pakistan' = 275.57498717854764
```

```python
from nltk.tokenize.treebank import TreebankWordDetokenizer
detok = TreebankWordDetokenizer()

def generate_tweet(model, num_words=20, seed=None):
    tokens = []
    for token in model.generate(num_words, text_seed=['<s>'], random_seed=seed):
        if token == '</s>':
            break
        tokens.append(token)
    return detok.detokenize(tokens)

print("Generated tweets:\n")
for s in range(1, 6):
    print(f"{s}. {generate_tweet(lm, num_words=20, seed=s)}")
```

```
Generated tweets:

1. : جتنے ریٹائرڈ آؤٹ کردیا جائےگا۔۔؟ بھٹو کی اہمیت اور پنجاب میں وردی اور ذمہ داری سابقہ گناہ
ہی اس
2. ڈیم کے اعمال بولتے ہیں میری فوج کا فکر و طالبات سے روشن ڈیجیٹل نشریاتی ادارے کے شواہد کے
جوانوں
3. : protest for rich in a 20-year-old friendship and early ‘ s rs600bil package for binance
in the ministry of
4. :
5. oooopsi krti or what say ufff!
```

Summary of the Task: Generating Tweets using N-Grams

1. Loading the data. The dataset *Large Random Tweets from Pakistan* (Kaggle, *adizafar/large-random-tweets-from-pakistan*) was downloaded with `kagglehub`. It contains a single CSV file with 202,202 tweets and 7 columns; the tweet text is stored in the `full_text` column. The file is not fully UTF-8 encoded, so it was read with `encoding_errors="ignore"` to skip the few invalid bytes while keeping the text intact.

2. Pre-processing. Each tweet was cleaned with regular expressions and the `emoji` library, in this order: remove `RT`, remove websites (`http\S+|www\.\S+`), remove mentions (`@\w+`), remove hashtags (`#\w+`), remove emojis (`emoji.replace_emoji`), collapse extra whitespace and lower-case the text. Tweets left with fewer than two words were dropped, which reduced the corpus from 202,202 to **171,454** tweets.

3. Building the model. Every cleaned tweet was tokenized with `word_tokenize`, padded and converted into everygrams using `padded_everygram_pipeline(2, ...)`, and used to fit an `MLE(2)` bigram language model. The resulting vocabulary contains **160,561** unique tokens.

4. Evaluating the model. Perplexity was computed on tweets taken from the corpus:

Test tweet	Perplexity
Tweet 1 (Urdu)	114.86
Tweet 2 (English)	90.75
Tweet 3 (Turkish)	6.58
An unseen bigram	inf

The lower the perplexity, the less "perplexed" the model is by the text. The unseen bigram returns infinity because the Maximum Likelihood Estimator applies no smoothing: any n-gram

that never appeared in the training corpus receives a probability of exactly zero. This is the main weakness of MLE models and the reason smoothing techniques are needed.

5. Probability of the bigram ("pakistan is").

- Count of `pakistan` = 13,649
- Count of `pakistan is` = 685
- P(is | pakistan) = 685 / 13,649 = **0.0502**

6. Perplexity of the word ("pakistan"). 275.57 — which corresponds to a unigram probability of about 1/275.57 ≈ 0.0036, i.e. roughly 0.36% of all tokens in the corpus are the word *pakistan*.

7. Generated tweets — observation. The generated sequences are locally plausible (each pair of consecutive words really occurred in the corpus) but globally incoherent, because a bigram model only remembers one previous word and has no understanding of the sentence as a whole. The generated text also mixes Urdu, English and Turkish, since the corpus itself is multilingual; a higher-order n-gram (trigram) or a language filter applied during pre-processing would produce more coherent output.